perplexity

Hello

Hello! How can I help with your Kirana Application/System project today?

from this above pdf what you got understood by the porject explain to me

The PDF describes **"Munshi" (or "Dukaan Voice")** — a voice-first, offline-capable POS and inventory app for Indian kirana stores. The shopkeeper can speak an order in Telugu, Hindi, English, or Hinglish, and the app creates the bill, deducts stock, and alerts them when items are running low. [1]

Core problem

Kirana owners currently calculate bills manually, remember stock levels, and track customer credit during busy hours. This leads to slow service, billing mistakes, stock mismatches, and missed reorder decisions. [1]

Existing apps often require typing, menu navigation, and reliable internet—barriers for many shopkeepers. The PDF therefore proposes a **conversation-first assistant** rather than another ordinary POS app. [1]

What the app does

The main workflow is:

1. The owner speaks: "Do kilo chawal, ek atta packet, teen Maggi."
2. The app identifies products, quantities, and units.
3. It calculates the bill and asks for confirmation.
4. After payment, it automatically reduces inventory.
5. It warns the owner when stock reaches the low-stock threshold.
6. It records customer credit/udhaar and generates daily sales reports. [1]

Example: if rice stock is 25 kg and a customer buys 2 kg, the app updates it to 23 kg and creates a permanent stock-ledger record. [1]

Main modules

Module	Purpose
Product catalog	Add products with names, aliases, barcode, price, unit, stock, threshold, and expiry. [1]
Voice billing	Convert spoken orders into itemized bills without typing. [1]
Inventory	Handle sales, purchases, damage, expiry, returns, and manual corrections. [1]
Stock ledger	Keep an auditable history of every stock change. [1]
Billing/payments	Support cash, UPI, card, and credit; optional printer and WhatsApp bill sharing. [1]
Customers/udhaar	Track customer balances, payments, purchase history, and credit limits. [1]
Reports	Show daily/weekly sales, profit estimates, low stock, dead stock, and reorder suggestions. [1]
Offline sync	Save all data locally first, then sync safely when internet returns. [1]

Key technical idea

The app is **offline-first**: every bill, product, and stock change is saved in local SQLite/Room storage immediately, then queued for cloud synchronization. The UI should read from the local database, so billing never depends on internet availability. [1]

Its voice pipeline is:

Microphone → Speech-to-text → Intent/entity extraction → Product matching → Confirmation → Local database update → Spoken confirmation → Sync queue. [1]

For example, “Aaj 10 kilo chawal aaya” should be parsed as:

```
{
  "intent": "ADD_STOCK",
  "product": "rice",
  "quantity": 10,
  "unit": "kg"
}
```

The app then confirms: “Chawal 10 kilo add karna hai. Confirm?” before changing stock. [1]

Most important rule

The PDF repeatedly emphasizes that inventory must be **auditable**. The app should never silently overwrite `current_stock`; every sale, purchase, damage, expiry, correction, or cancellation must create a `stock_ledger` entry showing the previous stock, new stock, reason, user, device, and timestamp. [1]

This builds trust because the owner can ask why the app says only 4 kg rice remains and see exactly which sales, purchases, or corrections caused it. [1]

Planned technology

Layer	Proposed choice
Mobile app	Flutter or Kotlin Android [1]
State management	Riverpod or Bloc [1]
Local database	SQLite/Drift or Room [1]
Speech recognition	Google STT / ML Kit / Indic ASR [1]
Voice response	Android Text-to-Speech [1]
NLP	Rule-based parser plus optional LLM fallback [1]
Barcode	ML Kit Barcode Scanning [1]
Backend	FastAPI or Node.js [1]
Cloud database	PostgreSQL [1]
Sync	Background worker / WorkManager with a sync queue [1]

MVP scope

The PDF recommends starting with a focused MVP rather than a full ERP:

- Product catalog with barcode, manual, and voice entry.
- Voice billing in Telugu, Hindi, English, and Hinglish.
- Automatic bill totals and stock deduction.
- Stock purchase, correction, damage, and expiry entries.
- Stock ledger and low-stock alerts.
- Daily sales summary.
- Offline storage and later cloud sync. [1]

It explicitly says to skip GST filing, full accounting, delivery management, supplier ordering, multi-store support, and advanced forecasting in the first version. [1]

What makes it unique

The proposed differentiator is not simply “voice billing.” It is a **Telugu/Hinglish-first voice munshi** that:

- Understands local product names such as “chawal,” “rice,” and “biyyam.”
- Works fully offline.
- Explains every stock change through a transparent ledger.
- Uses a large mic-first counter screen for rush hours.
- Combines inventory with udhaar tracking.
- Provides smart reorder suggestions based on sales velocity. [1]

Build roadmap

The document proposes roughly a 12-week plan:

- **Weeks 1–2:** Research, prototype, product catalog, database, stock ledger.
- **Weeks 3–4:** Billing, payments, stock deduction, customer credit.
- **Weeks 5–7:** Voice recognition, command parsing, voice billing and stock commands.
- **Weeks 8–9:** Backend, authentication, offline sync, conflict handling.
- **Weeks 10–12:** Reorder suggestions, expiry alerts, WhatsApp bills, printer, reports, and pilot testing.[1]

The success test is simple: a shopkeeper should be able to complete a real sale and update stock using only voice, without typing or confusion.[1]

⁂

and also it will be saying the if the item expiry date is soon it will be saying it

Yes — that is already part of the project plan. The app will store an **expiry date** for each product and show **expiry/near-expiry alerts**, so the shopkeeper knows which items should be sold or removed soon.[2]

How expiry alerts work

When adding a product or new stock, the owner enters the expiry date. The app compares today's date with that expiry date and flags products that are close to expiring or already expired.[2]

Example:

- Milk expiry date: 5 November 2026
- Today: 1 November 2026
- Warning threshold: 3 days
- App alert: "Milk expires in 4 days — sell it first."

The PDF includes "**Expiry tracking: Expiry date and near-expiry alerts**" as a product-catalog feature, and "**Expiry alerts: Items nearing expiry**" as an inventory feature.[2]

Voice support

The owner can also ask about expiring items by voice. A suitable command would be:

- "Kya expire hone wala hai?"
- "Expiring items sunao."
- "Milk ki expiry kab hai?"

The app can respond through text-to-speech, for example:

> “Doodh 4 din mein expire hoga. Bread 2 din mein expire hoga.”

This fits the existing voice pipeline: speech-to-text → intent/entity extraction → database query → spoken response. [2]

Where it appears in the app

Location	Expiry feature
Product screen	Enter and edit expiry date. [2]
Inventory screen	Separate **Expiry tab/screen** for near-expiry and expired items. [2]
Home/counter screen	Non-blocking warning banner for urgent expiry items. [2]
Reports	Expiry report showing items nearing expiry. [2]
Stock removal	Record expired stock as an `EXPIRY` movement in the stock ledger. [2]

Important design rule

If the owner marks an item as expired or removes it, the app must create a stock-ledger entry with `movement_type = EXPIRY`, the quantity removed, previous stock, new stock, and reason. This keeps inventory accurate and auditable. [2]

For better accuracy, especially for dairy or perishable goods, the PDF also suggests optional **batch/lot tracking**, so different batches of the same product can have different expiry dates. [2]

⁂

list me all technical features that need to included in this system

Here is the complete list of technical features your **Munshi / Voice-First Kirana POS** system should include, based on the PDF. The system is an offline-first Flutter app with voice billing, inventory auditing, customer credit, reports, and later cloud sync. [3]

1. App foundation

- Flutter Android app with a feature-first folder structure.
- Riverpod or Bloc for state management.
- GoRouter for navigation.
- Drift/SQLite local database.
- Repository pattern: UI must not directly access the database or API.
- Local database streams so screens update instantly from local data.
- UUIDs for local primary keys.

- ISO 8601 UTC timestamps.
- Large-button, counter-friendly UI with 48dp minimum buttons and 16sp minimum text.
- Bottom navigation: Home/Counter, Products, Inventory, Reports, Settings. [3]

2. Authentication and shop setup

- Owner signup/login using phone number and OTP.
- JWT access and refresh tokens.
- Secure token storage using Flutter Secure Storage.
- Owner/admin and helper/cashier roles.
- Helper permissions with restrictions on deleting products or approving large corrections.
- Shop profile: name, owner, phone, address, language, currency.
- Language preference: Telugu, Hindi, English, Hinglish.
- Optional GST settings, kept disabled for simple shops.
- Demo mode with sample products.
- Local encrypted backup and optional cloud backup. [3]

3. Product catalog

- Add, edit, archive, and search products.
- Product fields: name, brand, category, barcode, unit, purchase price, selling price, stock, expiry date, image, and low-stock threshold.
- Product aliases so “chawal,” “rice,” and “biyyam” map to the same item.
- Product categories such as groceries, snacks, dairy, beverages, toiletries, and household items.
- Barcode scanning using ML Kit for EAN/UPC/QR codes.
- Product photo storage for visual identification.
- Product variants such as 1 kg, 5 kg, packet, bottle, and dozen.
- Purchase and selling price history.
- Favorite/fast items pinned to the billing screen.
- Bulk CSV/Excel import and export.
- Soft delete/archive so historical records remain available. [3]

4. Voice assistant

This is the core differentiator. The voice pipeline should be:

Microphone → Speech-to-text → Text normalization → Intent classification → Entity extraction → Product matching → Confirmation → Action execution → Text-to-speech response. [3]

Required voice capabilities:

- Voice billing: “Do kilo chawal, ek atta packet, teen Maggi.”
- Stock addition: “Aaj 10 kilo chawal aaya.”
- Stock removal/damage: “Chawal 2 kilo kharab ho gaya.”
- Stock query: “Chawal kitna hai?”
- Low-stock query: “Kya khatam hone wala hai?”
- Expiry query: “Kya expire hone wala hai?”
- Stock correction: “Chawal 8 kilo karo.”
- Undo: “Last entry cancel karo.”
- Sales report: “Aaj kitna bikri hua?”
- Credit query: “Ramesh ka udhaar kitna hai?”
- Spoken confirmation before stock changes, bill payment, cancellation, and credit sale.
- Multilingual recognition for Telugu, Hindi, English, and Hinglish.
- Number-in-words handling: “do,” “teen,” “paanch,” “aadha.”
- Unit handling: kg, gram, litre, ml, packet, piece, dozen, bag, carton, and quintal.
- Product alias matching and fuzzy matching.
- Clarification when multiple products match.
- Low-confidence repeat request.
- Manual large-button fallback when speech recognition fails.
- Swappable STT and TTS providers. [3]

5. Billing and payments

- Create bills by voice, barcode, product search, favorites, or category browsing.
- Add, edit, remove, and change quantity/unit of bill items.
- Automatic line total, subtotal, discount, rounding, and final total.
- Flat, percentage, and item-level discounts.
- Payment modes: cash, UPI, card, credit/udhaar, and mixed payment.
- Sequential bill numbering.
- Bill preview with large readable total.
- Bill cancellation with mandatory reason.
- Full or partial return/refund with stock restoration.
- Daily closing summary for cash, UPI, credit, sales, and stock changes.
- Optional Bluetooth thermal printer support.
- Optional WhatsApp bill sharing as text, PDF, or image.
- Optional GST-ready fields, without forcing GST invoicing in the MVP. [3]

6. Inventory and stock control

- Automatic stock deduction when a bill is confirmed.
- Stock increase on purchase or supplier delivery.
- Manual stock correction with a mandatory reason.
- Damage, expiry, theft, and loss entries.
- Sale return and supplier return handling.
- Immutable stock ledger for every stock movement.
- Movement types: `OPENING_STOCK`, `PURCHASE`, `SALE`, `SALE_RETURN`, `DAMAGE`, `EXPIRY`, `MANUAL_FIX`, and `SUPPLIER_RETURN`.
- Low-stock alerts based on `current_stock <= low_stock_threshold`.
- Out-of-stock list.
- Expiry alerts and near-expiry list.
- Expiry date and optional batch/lot tracking for perishables.
- Physical stock-count mode comparing app stock with actual stock.
- Stock valuation at purchase and selling price.
- Supplier management with contact details, purchase history, and pending payable amount.
- Optional purchase orders/reorder list.
- Owner approval for large helper-made stock corrections.
- Transactional stock updates: never change `current_stock` without creating a ledger entry. [3]

7. Customers and udhaar

- Add and search customers by name or phone.
- Customer profile with name, phone, address, and notes.
- Credit/udhaar ledger: credit given, payments received, and balance.
- Credit sale linked to a bill.
- Credit payment recording.
- Optional customer credit limit and warning when exceeded.
- Customer purchase history and frequently bought items.
- WhatsApp/SMS payment-reminder drafts.
- Voice credit query. [3]

8. Reports and insights

- Daily, weekly, and monthly sales reports.
- Number of bills and cash/UPI/credit split.
- Top-selling and slow-moving products.
- Profit estimate using selling price minus purchase price.
- Current stock report and stock value.
- Low-stock report.
- Expiry report.
- Dead-stock report for products with no sales for N days.
- Credit report and outstanding udhaar.
- Expense entries for rent, electricity, transport, and supplier payments.
- Reorder suggestions based on sales velocity and current stock.
- CSV/PDF report export.
- Voice reports: “Aaj ka report sunao.”[3]

9. Offline-first architecture

- Local database is the source of truth while offline.
- Every product, bill, stock change, and customer update saves locally first.
- Every unsynced change enters a `sync_queue`.
- Background sync using WorkManager.
- Sync on app open, network reconnect, and periodic schedule.
- Batch upload of pending records.
- Retry with exponential backoff.
- Clear sync states: `PENDING`, `SYNCING`, `DONE`, `FAILED`, and `CONFLICT`.
- Network status indicator: “Offline — will sync later.”
- Adjustment-based stock sync, not absolute stock overwrite.
- Conflict rules: bills and stock ledger are append-only; product details use last-write-wins; manual corrections require owner confirmation on conflict.
- Multi-device support for owner and helper phones. [3]

10. Backend and APIs

- FastAPI or Node.js backend.
- PostgreSQL cloud database.
- JWT authentication and role-based authorization.
- REST APIs for products, stock, bills, customers, suppliers, reports, and voice interpretation.

- Product APIs: create, read, update, delete, search, and bulk import.
- Stock APIs: purchase, sale, damage, correction, ledger, low-stock, and valuation.
- Billing APIs: create bill, get bills by date, cancel bill, and process return.
- Voice API: accepts transcript and returns structured intent, items, quantities, confidence, and total.
- Cloud storage for product images and bill PDFs.
- Background jobs/workers for sync and retry handling.
- OpenAPI documentation with success and error responses. [3]

11. Database tables

The system needs at least these tables:

Table	Main purpose
`products`	Product details, aliases, barcode, prices, stock, threshold, expiry. [3]
`categories`	Product category management. [3]
`stock_ledger`	Immutable audit history of every stock change. [3]
`bills`	Bill header, total, payment mode, credit, status, sync state. [3]
`bill_items`	Individual products, quantities, units, prices, and line totals. [3]
`customers`	Customer details and udhaar balance. [3]
`suppliers`	Supplier details and pending payable amount. [3]
`sync_queue`	Pending local operations waiting for cloud sync. [3]
`users`	Owner/helper accounts, roles, and authentication data. [3]
`expenses`	Shop expense records. [3]

12. Security and privacy

- HTTPS for all API communication.
- Encrypted sensitive local data.
- Secure storage for authentication tokens.
- Role-based access control.
- Audit fields on every stock change: user, device, timestamp, movement type, previous stock, new stock, and reason.
- Do not retain raw voice audio unless the user opts in.
- Clear disclosure of cloud backup and voice processing.
- Account deletion and business-data export.
- No selling of shopkeeper data. [3]

13. Testing features

- Unit tests for price, discount, rounding, and unit conversion.
- Unit tests for product alias matching.
- Unit tests for stock sale, purchase, correction, damage, and expiry.
- Tests proving every stock change creates exactly one ledger entry.
- Tests for bill creation reducing stock.
- Tests for bill cancellation restoring stock.
- Tests for offline operations entering the sync queue.
- Integration tests for network reconnect and sync completion.
- Widget tests for billing screen, low-stock alerts, mic button, and payment buttons.
- Voice tests using Telugu, Hindi, English, and Hinglish samples.[3]

14. Advanced/pilot features

These can come after the core MVP:

- Reorder prediction from sales velocity.
- Photo-based product onboarding from shelf photos or supplier bills.
- Batch/lot-level expiry tracking.
- WhatsApp bill sharing.
- Thermal printer integration.
- Barcode scanner hardware integration.
- Weighing-scale integration.
- Owner/helper role-based access.
- Cloud backup and multi-device sync.
- Improved Telugu/Hinglish recognition using real pilot data.[3]

MVP priority

For the first working version, build these first:

1. Product catalog and barcode support.
2. Local database and stock ledger.
3. Manual stock add/remove/correction.
4. Billing with automatic stock deduction.
5. Customer udhaar.
6. Low-stock and expiry alerts.
7. Voice billing and stock commands.
8. Offline storage and sync queue.

9. Daily sales report. [3]

The most critical non-negotiable technical rule is: **never directly update `current_stock` without creating a `stock_ledger` entry inside a transaction.** This protects inventory accuracy even when voice recognition, helper entry, or sync conflicts cause mistakes. [3]

⁂

and also list me the functionality how i need to create and to be included here

Here are the **functionalities you need to create**, organized as user-facing workflows. Each one explains what the shopkeeper does, what the app must do, and what data it must save. [4]

1. Shop onboarding

Function: Set up the shop for the first time.

- Create owner account using phone number and OTP.
- Enter shop name, address, phone number, preferred language, and currency.
- Select language: Telugu, Hindi, English, or Hinglish.
- Create helper/cashier accounts with limited permissions.
- Add opening stock by manual entry, barcode scan, voice, photo, or CSV/Excel import.
- Enable demo mode with sample products for testing.
- Set up local encrypted backup and optional cloud backup. [4]

Data saved: Shop profile, user accounts, roles, language preference, products, and opening stock. [4]

2. Product management

Function: Maintain the list of items sold in the shop.

- Add a product manually with name, brand, category, barcode, unit, purchase price, selling price, stock, expiry date, and low-stock threshold.
- Add product aliases so “chawal,” “rice,” and “biyyam” all identify the same product.
- Scan a barcode to find or create a product.
- Add a product photo for easy identification.
- Search products by name, alias, barcode, or category.
- Edit product price, stock threshold, unit, and expiry date.
- Archive/hide a product without deleting its history.
- Mark products as favorites for fast billing.
- Import/export products using CSV or Excel. [4]

Data saved: Product details, aliases, barcode, category, prices, current stock, threshold, expiry date, image, and active status.[4]

3. Voice billing

Function: Create a customer bill by speaking naturally.

Flow:

1. Shopkeeper taps the large microphone button.
2. Shopkeeper says: “Do kilo chawal, ek atta packet, teen Maggi.”
3. App converts speech to text.
4. App identifies product, quantity, and unit.
5. App matches the spoken name using aliases and fuzzy matching.
6. App shows the item list and speaks the total: “2 kg rice ₹120, 1 atta packet ₹240. Total ₹360. Confirm?”
7. Shopkeeper confirms.
8. App creates the bill and deducts stock.[4]

Required sub-functions:

- Speech-to-text conversion.
- Text normalization for local words and numbers.
- Intent detection: create bill, add stock, check stock, etc.
- Entity extraction: product, quantity, unit, language.
- Product matching through exact name, alias, fuzzy search, or barcode.
- Voice confirmation before final action.
- Text-to-speech response.
- Manual fallback if voice recognition fails.[4]

Data saved: Bill, bill items, product price snapshot, total, payment mode, and stock-ledger entries.[4]

4. Manual billing

Function: Create a bill without voice when needed.

- Search a product or scan its barcode.
- Add products from favorites or category list.
- Enter or edit quantity and unit.
- Remove an item from the cart.
- Apply flat, percentage, or item-level discount.
- Automatically calculate subtotal, discount, and final total.

- Select payment mode: cash, UPI, card, credit/udhaar, or mixed.
- Generate a sequential bill number.
- Show a large, readable bill preview.
- Print the bill using a Bluetooth thermal printer.
- Share the bill through WhatsApp as text, PDF, or image.
- Cancel a bill with a mandatory reason.
- Process full or partial returns/refunds. [4]

Data saved: Bill header, bill items, discount, total, payment mode, paid amount, credit amount, status, and stock changes. [4]

5. Stock purchase

Function: Add new stock when supplies arrive.

Flow:

1. Shopkeeper says: “Aaj 10 kilo chawal aaya,” or opens Purchase Stock.
2. App asks for product, quantity, unit, purchase price, supplier, and expiry date.
3. Shopkeeper confirms.
4. App increases stock.
5. App updates the latest purchase price.
6. App creates a `PURCHASE` ledger entry. [4]

Formula:
`new_stock = current_stock + purchased_quantity` [4]

Data saved: Product ID, quantity, unit, purchase price, supplier, expiry date, previous stock, new stock, timestamp, user, device, and sync status. [4]

6. Stock sale deduction

Function: Reduce inventory automatically after every confirmed sale.

- When a bill is confirmed, subtract each sold quantity from product stock.
- Warn if the sale would make stock negative.
- Create a `SALE` stock-ledger entry.
- Trigger a low-stock alert if stock falls to or below the threshold.
- If an item is removed before payment, reverse only that item.
- If a completed bill is cancelled, restore stock through a `SALE_RETURN` entry instead of silently editing the sale. [4]

Formula:
`new_stock = current_stock - quantity_sold` [4]

7. Stock correction

Function: Fix wrong stock after physical counting, damage, expiry, or loss.

- Shopkeeper says: “Chawal 8 kilo karo,” or opens Adjust Stock.
- App shows current stock, entered actual stock, and the difference.
- App requires a reason: damage, counting mistake, theft, expiry, or other.
- App asks for confirmation.
- App sets the correct stock and creates a `MANUAL_FIX` ledger entry.
- For damage or expiry, create `DAMAGE` or `EXPIRY` entries with the quantity removed.[4]

Data saved: Previous stock, new stock, difference, reason, user, device, timestamp, and sync status.[4]

8. Stock checking and alerts

Function: Help the owner know what is available, low, or expiring.

- Check stock by voice: “Chawal kitna hai?”
- Check low stock: “Kya khatam hone wala hai?”
- Check expiring items: “Kya expire hone wala hai?”
- Show low-stock items when `current_stock <= low_stock_threshold`.
- Show out-of-stock items.
- Show near-expiry and expired items.
- Show a non-blocking warning banner on the Home screen.
- Provide reorder suggestions based on sales velocity and current stock.[4]

9. Customer and udhaar management

Function: Track regular customers and pending credit.

- Add a customer with name, phone, address, and notes.
- Search customers by name or phone.
- Create a credit sale and link it to the customer.
- Record credit payments.
- Show outstanding udhaar balance.
- Show customer purchase history and frequently bought items.
- Set an optional credit limit and warn when it is crossed.
- Create WhatsApp/SMS payment-reminder drafts.
- Ask by voice: “Ramesh ka udhaar kitna hai?”[4]

Data saved: Customer profile, credit balance, credit limit, bills, payments, and purchase history.[4]

10. Supplier management

Function: Track where stock comes from and what is payable.

- Add supplier name, phone, and address.
- Link purchases to a supplier.
- View supplier purchase history.
- Track pending amount payable to the supplier.
- Create a simple reorder/purchase list.
- Record return-to-supplier stock movements. [4]

Data saved: Supplier profile, purchase records, pending amount, and supplier-return ledger entries. [4]

11. Reports

Function: Give the owner a daily business summary.

- Daily, weekly, and monthly sales.
- Total bills and cash/UPI/credit split.
- Top-selling and slow-moving products.
- Profit estimate from selling price minus purchase price.
- Current stock and stock valuation.
- Low-stock report.
- Expiry report.
- Dead-stock report for products with no sales for N days.
- Credit/udhaar report.
- Expense entry for rent, electricity, transport, and supplier payments.
- CSV/PDF export.
- Voice report: “Aaj ka report sunao.” [4]

12. Offline operation

Function: Keep the app working without internet.

- Save every product, bill, stock change, and customer update locally first.
- Read UI data from local database streams.
- Add every unsynced change to a sync queue.
- Show a clear “Offline — will sync later” indicator.
- Sync automatically when internet returns, on app open, or through background sync.
- Upload changes in batches.
- Retry failed sync with exponential backoff.

- Sync stock as adjustments, not absolute stock values, to avoid multi-device overwrites.
- Handle conflicts: bills and ledger are append-only; product details use last-write-wins; manual corrections require owner confirmation on conflict. [4]

13. Security and roles

Function: Protect shop data and control access.

- Owner/admin can manage products, helpers, reports, and corrections.
- Helper/cashier can bill and make limited stock entries.
- Helper cannot delete products or approve large corrections.
- Require owner approval for large stock adjustments.
- Use HTTPS, encrypted local data, secure token storage, and role-based permissions.
- Record user ID, device ID, timestamp, and reason for every stock change. [4]

14. Expiry tracking

Function: Prevent losses from expired products.

- Enter expiry date when adding a product or purchase.
- Show near-expiry products in a dedicated Expiry screen/tab.
- Send alerts before expiry.
- Allow the owner to mark stock as expired.
- Create an `EXPIRY` stock-ledger entry with reason and quantity.
- Optionally track batch/lot numbers for dairy and perishable goods. [4]

Build order

Create the functionality in this sequence:

1. Flutter project, navigation, theme, and local database.
2. Product catalog and barcode support.
3. Stock ledger, purchase, damage/expiry, and manual correction.
4. Low-stock, out-of-stock, and expiry screens.
5. Billing, discounts, payments, and automatic stock deduction.
6. Bill cancellation and returns.
7. Customers and udhaar.
8. Voice billing and voice stock commands.
9. Reports.
10. Offline sync queue and backend/cloud sync.
11. WhatsApp bills, printer, reorder suggestions, and pilot features. [4]

The minimum “working product” is reached when the owner can add products, record purchases, create bills, deduct stock automatically, correct stock with a reason, see low-stock/expiry alerts, and use the app fully offline. [4]

⁂

i want to build using google ai studio so give me the documents or what ever that needs to included while building a product or a software

For **Google AI Studio**, you should prepare a small documentation pack and paste it as project context/system instructions. AI Studio works best when you give it a clear product definition, exact data rules, screen list, voice-command rules, and phased build prompts—not one vague “build an app” request. The PDF already recommends a documentation-first approach with a README, product requirements, functional specification, architecture, database schema, API spec, voice spec, UI spec, offline-sync spec, security, testing, and roadmap. [5]

Documents to prepare

Create these files in a Google Drive folder or paste them into AI Studio as context:

Document	What it contains
`README.md`	One-page project summary, core rules, tech stack, MVP scope. [5]
`01-product-requirements.md`	Problem, users, goals, non-goals, success metrics. [5]
`02-functional-specification.md`	Every module and user workflow. [5]
`03-technical-architecture.md`	App layers, folder structure, data flow, offline-first rules. [5]
`04-database-schema.md`	Tables, fields, relationships, ledger rules. [5]
`05-api-specification.md`	Backend endpoints, request/response examples, errors. [5]
`06-voice-assistant-spec.md`	Speech pipeline, intents, entities, aliases, confirmation rules. [5]
`07-ui-ux-specification.md`	Screens, layout, colors, accessibility, counter-mode design. [5]
`08-offline-sync-spec.md`	Local-first writes, sync queue, conflict handling. [5]
`09-security-privacy.md`	Authentication, roles, encryption, audit rules. [5]
`10-testing-plan.md`	Unit, widget, integration, and voice test cases. [5]
`11-development-roadmap.md`	Phase-wise build order and definition of done. [5]
`12-ai-studio-prompts.md`	Exact prompts to give Google AI Studio for each phase. [5]

1. README / project brief

Paste this first in Google AI Studio:

```
Project name: Munshi — Voice-First Kirana POS

Munshi is an offline-first Android POS and inventory app for Indian kirana stores.
The shopkeeper can speak in Telugu, Hindi, English, or Hinglish to create bills,
update stock, check inventory, manage customer udhaar, and see low-stock or
expiry alerts.

Core principles:
1. Offline-first: save every action locally before cloud sync.
2. Voice-first: minimize typing at the counter.
3. Auditable inventory: every stock change must create a stock_ledger entry.
4. Simple UI: large mic button, large buttons, spoken confirmation.
5. Telugu/Hinglish-first: support local product names and mixed speech.

MVP includes:
- Product catalog
- Barcode scanning
- Billing
- Automatic stock deduction
- Stock purchase, damage, expiry, and correction
- Stock ledger
- Voice billing and stock commands
- Low-stock and expiry alerts
- Customer udhaar
- Daily sales report
- Offline storage and later sync

Do not include in MVP:
- GST filing
- Full accounting
- Delivery management
- Multi-store franchise support
- Advanced forecasting
```

This matches the PDF's core product definition and MVP scope. [5]

2. Product requirements document

Include:

```
Problem:
Kirana shopkeepers manually calculate bills, remember stock, and track customer
credit. During rush hours, this causes slow service, billing errors, and stock
mismatches.

Target users:
- Kirana store owner
- Store helper/family member
- Small grocery/general store in India
```

```
Primary goals:
- Create a bill in under 15 seconds
- Reduce typing to near zero
- Automatically update stock after every sale
- Detect low stock and expiry risk
- Work without internet
- Support Telugu, Hindi, English, and Hinglish

Success metrics:
- Bill creation under 15 seconds
- Voice command success above 90%
- Stock mismatch below 5%
- At least 80% of bills created without typing
- 30-day shop retention above 60%
```

These goals and metrics come directly from the project specification. [5]

3. Functional specification

Give AI Studio the module list:

```
Modules:
1. Authentication: owner OTP login, helper login, role-based access.
2. Shop setup: shop profile, language, currency, opening stock, backup.
3. Product catalog: add/edit/archive products, aliases, barcode, categories,
   prices, units, stock, threshold, expiry, images, CSV import/export.
4. Voice assistant: billing, stock add/remove/query/correction, low-stock,
   expiry, sales report, credit query, undo, spoken confirmation.
5. Billing: voice/barcode/manual item add, discounts, totals, payment modes,
   bill number, cancellation, returns, daily closing.
6. Inventory: purchase, sale deduction, damage, expiry, manual correction,
   stock ledger, low-stock, out-of-stock, expiry, physical count, valuation.
7. Customers/udhaar: customer profiles, credit sale, payments, balance,
   purchase history, reminders.
8. Suppliers: supplier profile, purchases, pending payable, returns.
9. Reports: daily/weekly/monthly sales, profit, stock, low stock, expiry,
   dead stock, credit, expenses, CSV/PDF export, voice reports.
10. Offline sync: local database first, sync queue, background sync,
    adjustment-based stock sync, conflict handling.
```

This is the functional scope described in the PDF. [5]

4. Technical architecture

Tell AI Studio the exact architecture:

```
Architecture:

UI Layer
  ↓
Riverpod Controllers / Notifiers
  ↓
Domain Services / Use Cases
```

```
  ↓
Repositories
  ↓
Local Drift/SQLite Database + Sync Queue
  ↓
Backend API / PostgreSQL (later)

Rules:
- UI reads only from local database streams.
- All writes save locally first.
- UI must never wait for a network request.
- Repositories are the only layer allowed to access database or API.
- Business logic must be testable without Flutter widgets.
- No direct HTTP calls from UI code.
- No direct SQL queries from UI code.
```

The PDF specifies Flutter, Riverpod, Drift/SQLite, GoRouter, ML Kit, speech services, FastAPI, and PostgreSQL. [5]

5. Database schema

Include this exact data model:

```
Tables:

products:
product_id, name, aliases, barcode, category_id, unit,
purchase_price, selling_price, current_stock,
low_stock_threshold, expiry_date, image_path,
is_active, created_at, updated_at, synced

categories:
category_id, name

stock_ledger:
entry_id, product_id, movement_type, quantity_change,
previous_stock, new_stock, reason, bill_id, supplier_id,
user_id, device_id, created_at, synced

bills:
bill_id, bill_number, customer_id, subtotal, discount,
total, payment_mode, paid_amount, credit_amount,
status, created_at, synced

bill_items:
bill_item_id, bill_id, product_id, product_name,
quantity, unit, unit_price, line_total

customers:
customer_id, name, phone, address, credit_balance,
credit_limit, created_at

suppliers:
supplier_id, name, phone, address, pending_amount
```

```
sync_queue:
queue_id, entity_type, entity_id, operation, payload,
created_at, retry_count, status

Movement types:
OPENING_STOCK, PURCHASE, SALE, SALE_RETURN,
DAMAGE, EXPIRY, MANUAL_FIX, SUPPLIER_RETURN

Critical rule:
Never update current_stock directly.
Every stock-changing operation must create exactly one stock_ledger entry
inside a database transaction.
```

The PDF defines these tables and the non-negotiable ledger rule.[5]

6. Voice assistant specification

This is the most important document for Google AI Studio because it defines how the AI should understand speech.

```
Voice pipeline:
Audio capture
→ Speech-to-text
→ Text normalization
→ Intent classification
→ Entity extraction
→ Product matching
→ Confirmation
→ Action execution
→ Text-to-speech response

Supported intents:
CREATE_BILL: "Do kilo chawal, ek atta packet"
ADD_STOCK: "10 kilo chawal aaya"
REMOVE_STOCK: "2 kilo chawal kharab"
CHECK_STOCK: "Chawal kitna hai?"
LOW_STOCK: "Kya khatam hone wala hai?"
EXPIRY_CHECK: "Kya expire hone wala hai?"
CORRECT_STOCK: "Chawal 8 kilo karo"
CANCEL_LAST: "Last entry cancel karo"
SALES_REPORT: "Aaj kitna bikri hua?"
CREDIT_QUERY: "Ramesh ka udhaar kitna hai?"

Entity output format:
{
  "intent": "ADD_STOCK",
  "product": "rice",
  "quantity": 10,
  "unit": "kg",
  "language": "hinglish",
  "confidence": 0.93
}
```

```
Product aliases:
rice: ["chawal", "biyyam", "chaawal"]
wheat_flour: ["atta", "gehu ka atta"]
oil: ["tel", "cooking oil"]

Units:
kg, gram, litre, ml, piece, packet, dozen, bag, carton, quintal

Confirmation rules:
Always confirm stock addition, stock removal, stock correction,
bill payment, bill cancellation, and credit sale.

Failure rules:
- Low confidence: ask user to repeat.
- Multiple matches: show selectable options.
- Unknown product: offer to add product.
- No internet: use manual entry fallback.
```

The PDF defines this pipeline, intents, entity output, alias examples, and confirmation/failure behavior. [5]

7. UI/UX specification

```
Screens:
1. Home/Counter: huge mic button, current bill, running total,
   Cash/UPI/Credit buttons, barcode button, low-stock and expiry banners.
2. Products: search, category chips, product cards, add product, barcode scan.
3. Product Add/Edit: name, aliases, barcode, category, unit, prices,
   stock, threshold, expiry date, image.
4. Inventory: current stock, low stock, out of stock, expiry, add stock,
   correct stock, stock ledger, physical count.
5. Billing: item list, quantity editor, discount, total, payment selector,
   print/share buttons.
6. Customers: customer list, udhaar balance, payment history.
7. Reports: today/week/month, sales, profit, top products, low stock, credit.
8. Settings: shop profile, language, users, backup, sync status.

UI rules:
- One primary action per screen.
- Minimum button height 48dp.
- Minimum body font size 16sp.
- Large mic button.
- Voice feedback for totals and stock changes.
- Clear offline status.
- Telugu-first labels with English fallback.
- Avoid nested navigation during billing.
```

These UI rules are specified for a busy, low-typing counter environment. [5]

8. Offline sync specification

```
Offline rules:
1. Local database is the source of truth while offline.
2. Every create/update writes locally first.
3. Every unsynced change enters sync_queue.
4. Sync runs on app open, network reconnect, and periodic background sync.
5. Upload changes in batches.
6. Retry failed sync with exponential backoff.
7. Show sync states: PENDING, SYNCING, DONE, FAILED, CONFLICT.

Stock sync rule:
Never sync absolute stock values from multiple devices.
Sync stock adjustments:
{
  "product_id": "prod_123",
  "quantity_change": -2,
  "movement_type": "SALE",
  "device_id": "device_001",
  "timestamp": "2026-11-01T10:30:00Z"
}

Conflict rules:
- Bills: append-only.
- Stock ledger: append-only.
- Product details: last-write-wins using updated_at.
- Manual corrections: require owner confirmation on conflict.
```

This prevents stock corruption when the owner and helper use separate phones. [5]

9. Security and privacy

```
Security:
- Phone OTP login.
- JWT access and refresh tokens.
- Store tokens in secure storage.
- HTTPS for all API calls.
- Role-based access: owner and helper.
- Helper cannot delete products or approve large corrections.
- Encrypt sensitive local data.
- Do not store raw audio unless user opts in.
- Every stock change must record user_id, device_id, timestamp,
  movement_type, previous_stock, new_stock, and reason.
- Provide data export and account deletion.
```

The PDF includes these security, privacy, and auditability requirements. [5]

10. Testing plan

```
Unit tests:
- Product creation
- Alias matching
- Price and discount calculation
- Unit conversion
- Stock purchase
- Stock sale
- Stock correction
- Damage/expiry removal
- Low-stock detection
- Bill cancellation stock restoration

Integration tests:
- Create bill → stock reduces
- Cancel bill → stock restores
- Add purchase → stock increases
- Offline bill → enters sync queue
- Network reconnect → sync completes

Voice tests:
- "Do kilo chawal"
- "Biyyam 5 kg"
- "Ek packet atta"
- "Paanch Maggi"
- "Chawal 10 kilo aaya"
- "Chawal 2 kilo kharab"
- "Chawal kitna hai?"
- "Kya expire hone wala hai?"
- "Ramesh ka udhaar kitna hai?"

Acceptance criteria:
- Shopkeeper can complete a sale without typing.
- Stock updates automatically after bill confirmation.
- App works fully offline.
- No stock change occurs without a ledger entry.
- Voice confirmation occurs before stock corrections.
```

These are the test cases and acceptance criteria from the project plan.[5]

11. Development roadmap

```
Phase 1 — Core offline inventory:
Flutter setup, Drift database, product CRUD, barcode placeholder,
stock ledger, manual stock updates, low-stock alerts, tests.

Phase 2 — Billing:
Billing screen, cart, discounts, payment modes, bill creation,
automatic stock deduction, cancellation, customer credit, daily report.

Phase 3 — Voice MVP:
Mic UI, speech-to-text, intent parser, entity extractor, alias matching,
voice billing, stock commands, TTS confirmation, manual fallback.
```

```
Phase 4 — Sync:
FastAPI backend, PostgreSQL, JWT auth, sync queue, WorkManager,
adjustment-based sync, conflict handling, cloud backup.

Phase 5 — Pilot features:
Reorder suggestions, expiry/batch tracking, photo onboarding,
WhatsApp bills, printer, voice reports, owner/helper permissions.
```

This follows the PDF's 12-week implementation roadmap.[5]

Master prompt for Google AI Studio

Paste this as your first message after adding the documents:

```
You are a senior Flutter engineer and product architect building "Munshi,"
an offline-first, voice-first POS and inventory app for Indian kirana stores.

Before writing any code:
1. Read all project documents provided.
2. Summarize your understanding in maximum 10 bullets.
3. Create a detailed implementation plan for Phase 1 only.
4. List every file you will create or modify.
5. Ask me for confirmation before generating code.

Product:
Munshi helps kirana shopkeepers create bills, update inventory, track udhaar,
check low-stock and expiry items, and use voice commands in Telugu, Hindi,
English, and Hinglish.

Non-negotiable rules:
1. Use Flutter, Riverpod, Drift/SQLite, and GoRouter.
2. Use a feature-first folder structure.
3. The app must be offline-first.
4. UI must read from local database streams.
5. All writes must save locally before network sync.
6. Never directly update current_stock without creating a stock_ledger entry.
7. Every stock-changing operation must run inside a transaction.
8. A completed sale reduces stock.
9. A cancelled sale restores stock.
10. A purchase increases stock.
11. Damage, expiry, theft, and manual correction must create ledger entries
    with a reason.
12. Use UUIDs for local IDs.
13. Use ISO 8601 UTC timestamps.
14. Store quantities as double to support kg, gram, litre, packet, piece,
    dozen, and fractional quantities.
15. Do not implement backend, cloud sync, authentication, payments, GST,
    printer, or analytics unless the current phase explicitly asks for them.

Current task: Phase 1 only — Core offline inventory foundation.

Build:
1. Flutter project structure.
2. Riverpod setup.
```

```
3. Drift database.
4. GoRouter navigation.
5. Bottom navigation: Home/Counter, Products, Inventory, Reports, Settings.
6. Database tables: products, categories, stock_ledger, bills, bill_items,
   customers, suppliers, sync_queue.
7. Product list, add/edit, search, aliases, barcode field, archive.
8. Inventory: purchase entry, damage/expiry removal, manual correction
   with mandatory reason, stock ledger, low-stock screen, out-of-stock
   screen, expiry screen.
9. Low-stock warning banner on Home.
10. Unit tests for all stock-changing logic.

First show:
- Understanding summary
- Phase 1 implementation plan
- Files to create
- Questions or assumptions

Wait for my confirmation before writing code.
```

This mirrors the PDF's recommended master prompt: read documents, plan first, build only the current phase, and preserve the inventory-ledger safety rule.[5]

How to use it in AI Studio

1. Create a Google Drive folder named `munshi-kirana-pos`.
2. Add the 12 documents above, preferably as Markdown or Google Docs.
3. Open Google AI Studio and start a new chat.
4. Paste the **README/project brief** and **database schema** first.
5. Paste the **master prompt**.
6. Ask it to generate only Phase 1.
7. After testing Phase 1, use this follow-up:

```
Phase 1 is complete and tests pass.

Now implement Phase 2 only: Billing and customer credit.

Read the project documents again.

Requirements:
- Billing screen with product search, barcode placeholder, favorites,
  quantity editor, discounts, and payment modes.
- Bill creation must be transactional.
- Confirmed bills must reduce stock and create SALE ledger entries.
- Bill cancellation must restore stock and create SALE_RETURN entries.
- Credit sales must update customer credit balance.
- Add daily sales summary.
- Add unit and integration tests.
```

```
Do not implement voice, backend, sync, printing, or WhatsApp yet.
First show the plan and wait for confirmation.
```

The key is to build in phases and always require the plan before code. This prevents AI Studio from creating disconnected screens or corrupting stock data. [5]

⁂

give me the documents as to be downloaded and also give me mvp level like to bring it to users for real time work

I've created a complete downloadable documentation pack for building **Munshi** in Google AI Studio, plus a separate real-user MVP launch plan. The pack includes 14 Markdown documents covering product requirements, functionality, architecture, database, APIs, voice commands, UI, offline sync, security, testing, roadmap, and ready-to-paste AI Studio prompts. [6]

Download files

- **Full documentation ZIP:** `Munshi_Kirana_POS_Documentation.zip` [6]
- Individual documents: `README.md`, `01-product-requirements.md`, `02-functional-specification.md`, `03-technical-architecture.md`, `04-database-schema.md`, `05-api-specification.md`, `06-voice-assistant-spec.md`, `07-ui-ux-specification.md`, `08-offline-sync-spec.md`, `09-security-privacy.md`, `10-testing-plan.md`, `11-development-roadmap.md`, `12-ai-studio-prompts.md`, and `13-MVP-USER-LAUNCH-PLAN.md`. [6]

MVP for real users

For a first real-world launch, do **not** build everything at once. Launch with a reliable offline billing and inventory core, then add advanced features after shopkeeper feedback. The PDF recommends a focused MVP rather than a full ERP. [6]

Must-have launch features

1. Owner phone OTP login and helper role.
2. Shop profile and language selection.
3. Product catalog with manual add, barcode scan, aliases, price, unit, stock, threshold, and expiry date.
4. Opening-stock and purchase-stock entry.
5. Billing with search, barcode, favorites, quantity editor, discounts, and totals.
6. Automatic stock deduction after a confirmed bill.
7. Cash, UPI, and credit/udhaar payment.
8. Customer credit ledger and payment recording.
9. Bill cancellation and return with stock restoration.

10. Immutable stock ledger.
11. Manual stock correction with mandatory reason.
12. Damage and expiry stock removal.
13. Low-stock alerts.
14. Expiry/near-expiry alerts.
15. Daily sales summary.
16. Offline-first local database.
17. Basic cloud backup/sync.
18. Voice MVP for the most common commands. [6]

Voice commands for launch

Start with only high-value commands:

- “Do kilo chawal, ek atta packet” → create bill.
- “10 kilo chawal aaya” → add stock.
- “Chawal kitna hai?” → check stock.
- “Kya khatam hone wala hai?” → low-stock list.
- “Kya expire hone wala hai?” → expiry list.
- “Chawal 8 kilo karo” → correct stock.
- “Ramesh ka udhaar kitna hai?” → credit query. [6]

Every stock-changing voice command must ask for confirmation before saving, for example: “Chawal 10 kilo add karna hai. Confirm?” [6]

Do not include in first release

Avoid these initially because they increase complexity without being essential for a single-shop MVP:

- GST invoicing and tax filing.
- Full accounting.
- Delivery management.
- Multi-store/franchise support.
- Advanced demand forecasting.
- Complex supplier purchase orders.
- Automatic photo-based catalog creation. [6]

These can become paid add-ons or Phase 2/3 features after the core app is stable. [6]

Real-user pilot plan

1. Select **5–10 nearby kirana shops**.
2. Install the app and add **30–100 real products** per shop.
3. Train the owner/helper for **30–45 minutes**.
4. Run it alongside their current method for **7 days**.
5. Collect daily feedback and fix the top three usability issues.
6. Measure active usage after **30 days**.[6]

Launch success targets

Metric	Target
Bill creation time	Under 15 seconds [6]
Voice command success	Above 90% for common items [6]
Stock mismatch after daily count	Below 5% [6]
Bills created without typing	At least 80% [6]
Shops active after 30 days	Above 60% [6]

Launch readiness checklist

Before giving it to real shopkeepers, ensure:

- App installs and runs on low-end Android phones.
- Billing and stock updates work fully offline.
- Cancelled bills restore stock.
- Purchases increase stock.
- Every stock change appears in the ledger.
- Low-stock and expiry alerts work.
- Udhaar balances update correctly.
- Voice fallback/manual entry works when speech recognition fails.
- Backup and restore work.
- Owner/helper permissions work.[6]

The most important MVP rule remains: **never directly update** `current_stock` **without creating a** `stock_ledger` **entry inside a transaction.** This is what makes the inventory trustworthy for real shops.[6]

⁂

give me the master prompt that to be given to the google ai studio

Copy and paste this as your **first master prompt in Google AI Studio.** It tells the AI to plan first, build only Phase 1, and protect the inventory-ledger rules that are central to your Kirana POS. [7]

```
You are a senior Flutter engineer, UI/UX designer, and technical lead building
“Munshi”, an offline-first, voice-first POS and inventory app for Indian kirana
stores.

Your job is to build the app incrementally, safely, and exactly according to the
project requirements. Do not invent extra features, change the architecture, or
add dependencies unless the current task explicitly requires them.

Before writing any code:
1. Understand the complete product requirements below.
2. Summarize your understanding in maximum 10 bullets.
3. Create a detailed implementation plan for Phase 1 only.
4. List every file, screen, model, repository, service, and test you will create.
5. Ask me for confirmation before generating code.

# PRODUCT

Munshi helps kirana shopkeepers:
- Create customer bills quickly.
- Update inventory automatically after sales.
- Add purchased stock.
- Correct damaged, expired, or miscounted stock.
- Track customer udhaar/credit.
- Check low-stock and expiry items.
- Use voice commands in Telugu, Hindi, English, and Hinglish.

The app must be usable by a shopkeeper with little typing experience.
Voice is the primary input; typing, barcode scanning, and manual entry are
fallbacks.

# CORE PRODUCT RULES

1. Offline-first:
   - Every product, bill, stock change, and customer update must save locally first.
   - The UI must read from local database streams.
   - The UI must never wait for a network request to display core data.
   - Network sync is a background enhancement, not a requirement for billing or
     inventory updates.

2. Voice-first:
   - Provide a large microphone button on the Home/Counter screen.
   - Support Telugu, Hindi, English, and Hinglish commands.
   - Support local product names and aliases such as:
     chawal, rice, biyyam, atta, tel, etc.
   - Convert spoken commands into structured JSON before executing them.
   - Always confirm destructive, stock-changing, or financial actions by voice
     and on screen.
```

```
   - If confidence is low, ask the user to repeat.
   - If multiple products match, show selectable options.
   - Provide manual entry fallback when speech recognition fails.

3. Inventory integrity:
   - Never directly update current_stock without creating a stock_ledger entry.
   - Every stock-changing operation must run inside a database transaction.
   - Every stock_ledger entry must include:
     entry_id, product_id, movement_type, quantity_change, previous_stock,
     new_stock, reason, bill_id, supplier_id, user_id, device_id, created_at,
     synced.
   - A completed sale must reduce stock.
   - A cancelled sale must restore stock.
   - A purchase must increase stock.
   - Damage, expiry, theft, and manual correction must decrease or set stock
     only through a ledger entry with a reason.
   - Low-stock alert must appear when current_stock is less than or equal to
     low_stock_threshold.
   - Expiry alert must appear for products nearing or past expiry_date.

# TECH STACK

Use:
- Flutter
- Riverpod for state management
- Drift/SQLite for local offline database
- GoRouter for navigation
- Feature-first folder structure
- ML Kit Barcode Scanning for barcode support
- Speech-to-Text and Text-to-Speech for voice input/output
- FastAPI backend and PostgreSQL only in the later sync phase

Architecture:
UI Layer
  ↓
Riverpod Controllers / Notifiers
  ↓
Domain Services / Use Cases
  ↓
Repositories
  ↓
Local Drift/SQLite Database + Sync Queue
  ↓
Backend API / PostgreSQL (later)

Engineering rules:
- UI must not call Drift database or Dio/HTTP directly.
- UI must use Riverpod providers/controllers.
- Controllers must call domain services or repositories.
- Repositories are the only layer allowed to access local database or API.
- Use UUIDs for local primary keys.
- Use ISO 8601 UTC timestamps.
- Store quantities as double/REAL to support kg, gram, litre, ml, packet,
  piece, dozen, bag, carton, and fractional quantities.
- Store money as double for MVP, but centralize rounding logic.
- Do not add packages without asking me first.
```

- Do not change public function signatures or database schemas without
  explaining the migration plan.

DATABASE TABLES

Create these tables:

products:
product_id, name, aliases, barcode, category_id, unit, purchase_price,
selling_price, current_stock, low_stock_threshold, expiry_date, image_path,
is_active, created_at, updated_at, synced

categories:
category_id, name

stock_ledger:
entry_id, product_id, movement_type, quantity_change, previous_stock,
new_stock, reason, bill_id, supplier_id, user_id, device_id, created_at,
synced

bills:
bill_id, bill_number, customer_id, subtotal, discount, total, payment_mode,
paid_amount, credit_amount, status, created_at, synced

bill_items:
bill_item_id, bill_id, product_id, product_name, quantity, unit, unit_price,
line_total

customers:
customer_id, name, phone, address, credit_balance, credit_limit, created_at

suppliers:
supplier_id, name, phone, address, pending_amount

sync_queue:
queue_id, entity_type, entity_id, operation, payload, created_at,
retry_count, status

Stock movement types:
OPENING_STOCK, PURCHASE, SALE, SALE_RETURN, DAMAGE, EXPIRY, MANUAL_FIX,
SUPPLIER_RETURN

MVP SCOPE

The MVP must include:
- Product catalog
- Product aliases
- Barcode scanning
- Manual and voice product addition
- Opening stock
- Purchase stock entry
- Billing
- Automatic stock deduction
- Bill cancellation and returns
- Damage/expiry/loss entry
- Manual stock correction with mandatory reason

```
- Stock ledger
- Low-stock alerts
- Expiry alerts
- Customer udhaar/credit
- Daily sales summary
- Offline-first local database
- Voice billing and stock commands
- Basic cloud backup/sync later

Do NOT include in the MVP:
- GST filing
- Full accounting
- Delivery management
- Multi-store franchise management
- Advanced demand forecasting
- Complex supplier purchase orders

# CURRENT TASK: PHASE 1 ONLY

Build only Phase 1: Core offline inventory foundation.

Do not implement voice, backend, cloud sync, authentication, payments,
GST, printer integration, WhatsApp, or analytics yet.

Create:

1. Flutter project setup:
   - Clean feature-first folder structure.
   - Riverpod setup.
   - Drift/SQLite database setup.
   - GoRouter setup.
   - Theme with large fonts and large touch targets.
   - Bottom navigation with:
     Home/Counter, Products, Inventory, Reports, Settings.

2. Database:
   - Implement Drift tables for:
     products, categories, stock_ledger, bills, bill_items, customers,
     suppliers, sync_queue.
   - Add DAOs, repositories, type converters where needed, and migration setup.
   - Add synced, created_at, and updated_at fields where required.
   - Every stock-changing operation must create exactly one stock_ledger entry
     inside a transaction.

3. Product module:
   - Product list screen.
   - Add/edit product screen.
   - Product search.
   - Product category selection.
   - Product alias support.
   - Barcode field and barcode scanning placeholder.
   - Purchase price, selling price, current stock, unit, low-stock threshold,
     and expiry date.
   - Archive product.
   - Product repository and Riverpod providers.
```

```
4. Inventory module:
   - Add stock / purchase entry.
   - Remove stock for damage, expiry, or loss.
   - Manual stock correction with mandatory reason.
   - Stock ledger screen showing all movements.
   - Low-stock screen.
   - Out-of-stock screen.
   - Expiry screen.
   - Stock valuation report.
   - Physical stock count screen.

5. Low-stock and expiry alerts:
   - Show low-stock items when current_stock <= low_stock_threshold.
   - Show near-expiry and expired products.
   - Show a non-blocking warning banner on the Home screen.

6. Testing:
   - Write unit tests for:
     product creation, product alias matching, stock purchase, stock sale,
     stock correction, damage/expiry removal, stock ledger creation,
     low-stock detection, and expiry detection.
   - Ensure every stock-changing operation creates exactly one ledger entry.

# REQUIRED OUTPUT FORMAT

First, provide:
1. A short understanding summary in maximum 10 bullets.
2. A Phase 1 implementation plan.
3. A list of files you will create or modify.
4. Any questions or assumptions.

Wait for my confirmation.

After confirmation, implement Phase 1 in small, reviewable steps:
- Step 1: Project structure and dependencies.
- Step 2: Database schema and repositories.
- Step 3: Product module.
- Step 4: Inventory and stock ledger.
- Step 5: Low-stock and expiry alerts.
- Step 6: Tests.

For every step:
- Explain what you are implementing.
- Show only the files created or changed.
- Keep code clean, typed, and production-quality.
- Add meaningful error handling.
- Add comments only where business logic is non-obvious.
- Consider edge cases before saying the step is complete.

# DEFINITION OF DONE FOR PHASE 1

Phase 1 is complete only when:
- I can add a product manually.
- I can add stock for a product.
- I can record damage or expiry.
- I can manually correct stock with a reason.
```

```
- I can view the complete stock ledger.
- I can see low-stock, out-of-stock, and expiring products.
- Every stock change has an auditable ledger entry.
- The app works completely offline.
- All stock-related unit tests pass.
```

How to use it

1. Open **Google AI Studio** and start a new chat.
2. Paste this master prompt.
3. The AI should first show a plan and wait for your confirmation.
4. Approve only **Phase 1**.
5. After Phase 1 works, use this follow-up prompt:

```
Phase 1 is complete and tests pass.

Now implement Phase 2 only: Billing and customer credit.

Requirements:
- Billing screen with product search, barcode placeholder, favorites,
  quantity editor, discounts, and payment modes.
- Bill creation must be transactional.
- Confirmed bills must reduce stock and create SALE ledger entries.
- Bill cancellation must restore stock and create SALE_RETURN entries.
- Credit sales must update customer credit balance.
- Add daily sales summary.
- Add unit and integration tests.

Do not implement voice, backend, sync, printing, or WhatsApp yet.
First show the plan and wait for confirmation.
```

Then, after billing works:

```
Phase 2 is complete and tests pass.

Now implement Phase 3 only: Voice MVP.

Requirements:
- Mic button and audio recording UI.
- Speech-to-text service interface.
- Text-to-speech service interface.
- Intent classifier and entity extractor.
- Product alias matcher.
- Voice billing command.
- Voice stock add/remove/query/correction.
- Expiry query.
- Voice confirmation before stock changes.
- Manual fallback on low confidence.
- Unit tests for command parsing.
```

```
First show the plan and wait for confirmation.
```

The key protection is the instruction: **never directly update `current_stock` without creating a `stock_ledger` entry.** Keep that rule in every phase so voice mistakes, helper errors, or sync conflicts cannot silently corrupt inventory. [7]

⁂

now tell me how to proceed with these all documents that you have given and also this prompt

means we need to place the files in this and then giving this prompt in that same query is that good or else do you have other approach to go with

Yes—placing the documents in the same chat/project and then giving the master prompt is a good approach, but for **Google AI Studio** the better method is: **upload only the essential documents, keep the master prompt as the first instruction, and build one phase at a time.** Do not paste all 14 documents into one giant message if they exceed the context or make the AI lose focus. [8]

Best workflow for Google AI Studio

Step 1: Create a project folder

Make this structure on your computer or Google Drive:

```
munshi-kirana-pos/
├── README.md
├── 01-product-requirements.md
├── 02-functional-specification.md
├── 03-technical-architecture.md
├── 04-database-schema.md
├── 05-api-specification.md
├── 06-voice-assistant-spec.md
├── 07-ui-ux-specification.md
├── 08-offline-sync-spec.md
├── 09-security-privacy.md
├── 10-testing-plan.md
├── 11-development-roadmap.md
├── 12-ai-studio-prompts.md
└── 13-MVP-USER-LAUNCH-PLAN.md
```

This documentation-first structure is recommended in the PDF because explicit, scoped files prevent the AI from building disconnected screens or violating inventory rules. [8]

Step 2: Upload only Phase 1 documents

For the first Google AI Studio chat, upload these **five files only**:

1. `README.md`
2. `01-product-requirements.md`
3. `02-functional-specification.md`
4. `03-technical-architecture.md`
5. `04-database-schema.md`

Do **not** upload the backend/API, sync, voice, printer, WhatsApp, or launch-plan documents yet. They are for later phases. This keeps the AI focused on the offline inventory foundation.[8]

Step 3: Paste the master prompt

After uploading those files, paste the master prompt in the same chat. It should tell the AI to read the uploaded documents, summarize its understanding, create a Phase 1 plan, and wait for your approval before coding.[8]

Use this exact opening prompt:

```
I have uploaded the project documents for “Munshi”, an offline-first,
voice-first POS and inventory app for Indian kirana stores.

You are a senior Flutter engineer and technical lead.

Before writing any code:
1. Read and understand all uploaded documents.
2. Summarize your understanding in maximum 10 bullets.
3. Create a detailed implementation plan for Phase 1 only.
4. List every file, screen, model, repository, service, and test you will create.
5. Ask me for confirmation before generating code.

Build only Phase 1: Core offline inventory foundation.

Do not implement voice, backend, cloud sync, authentication, payments, GST,
printer, WhatsApp, or analytics yet.

Phase 1 must include:
1. Flutter project setup with Riverpod, Drift/SQLite, and GoRouter.
2. Feature-first folder structure.
3. Bottom navigation: Home/Counter, Products, Inventory, Reports, Settings.
4. Database tables: products, categories, stock_ledger, bills, bill_items,
   customers, suppliers, sync_queue.
5. Product list, add/edit, search, aliases, barcode field, archive.
6. Inventory: purchase entry, damage/expiry removal, manual correction with
   mandatory reason, stock ledger, low-stock screen, out-of-stock screen,
   expiry screen.
7. Low-stock and expiry warning banner on Home.
8. Unit tests for every stock-changing operation.

Non-negotiable rules:
```

```
- The app must be offline-first.
- UI must read from local database streams.
- All writes must save locally first.
- Never directly update current_stock without creating a stock_ledger entry.
- Every stock-changing operation must run inside a database transaction.
- A completed sale reduces stock.
- A cancelled sale restores stock.
- A purchase increases stock.
- Damage, expiry, theft, and manual correction must create ledger entries
  with a reason.
- Use UUIDs for local IDs.
- Use ISO 8601 UTC timestamps.
- Store quantities as double to support kg, gram, litre, packet, piece,
  dozen, and fractional quantities.

First show:
1. Understanding summary
2. Phase 1 implementation plan
3. Files to create
4. Questions or assumptions

Wait for my confirmation before writing code.
```

Step 4: Approve and build step by step

When the AI gives the plan, check these points before saying "yes":

- It is building **only Phase 1**.
- It is not adding voice, backend, sync, payments, or GST.
- It includes `stock_ledger` for every stock change.
- It uses transactions for billing and stock updates.
- It creates low-stock and expiry screens.
- It includes tests.

Then reply:

```
Confirmed. Implement Phase 1 step by step.
Start with Step 1 only: Flutter project structure and dependencies.
After each step, show the files created or changed and wait for my review.
```

This step-by-step approach matches the PDF's recommended build order: project setup → database → product module → inventory/ledger → alerts → tests. [8]

Step 5: Test Phase 1 before continuing

After Phase 1, manually verify:

- You can add a product.
- You can add purchase stock.

- You can record damage or expiry.
- You can correct stock with a reason.
- You can see the complete stock ledger.
- You can see low-stock, out-of-stock, and expiring products.
- Every stock change creates a ledger entry.
- The app works without internet. [8]

Only after these work should you move to Phase 2.

Step 6: Start a fresh chat for each major phase

For Google AI Studio, the cleanest approach is:

Phase	Upload documents	Prompt
Phase 1: Inventory core	README, PRD, functional spec, architecture, database schema	Master prompt above
Phase 2: Billing + udhaar	Previous docs + testing plan	Phase 2 prompt
Phase 3: Voice MVP	Previous docs + voice assistant spec + UI spec	Phase 3 prompt
Phase 4: Sync/backend	Previous docs + API spec + offline-sync spec + security doc	Phase 4 prompt
Phase 5: Pilot features	All documents + MVP launch plan	Phase 5 prompt

This avoids context overload and helps the AI focus on one deliverable at a time. The PDF similarly recommends implementing modules in a fixed sequence rather than building everything simultaneously. [8]

Phase 2 prompt

After Phase 1 is tested, upload `10-testing-plan.md` too and use:

```
Phase 1 is complete and tested.

Now implement Phase 2 only: Billing and customer credit.

Read the uploaded documents again.

Requirements:
- Billing screen with product search, barcode placeholder, favorites,
  quantity editor, discounts, and payment modes.
- Bill creation must be transactional.
- Confirmed bills must reduce stock and create SALE ledger entries.
- Bill cancellation must restore stock and create SALE_RETURN entries.
- Credit sales must update customer credit balance.
- Add daily sales summary.
- Add unit and integration tests.

Do not implement voice, backend, sync, printing, or WhatsApp yet.

First show:
1. Implementation plan
```

```
2. Files to create or modify
3. Questions or assumptions

Wait for my confirmation before writing code.
```

Phase 3 prompt

After billing works, upload `06-voice-assistant-spec.md` and `07-ui-ux-specification.md`, then use:

```
Phase 2 is complete and tested.

Now implement Phase 3 only: Voice MVP.

Requirements:
- Mic button and audio recording UI.
- Speech-to-text service interface.
- Text-to-speech service interface.
- Intent classifier and entity extractor.
- Product alias matcher.
- Voice billing command.
- Voice stock add/remove/query/correction.
- Expiry query.
- Voice confirmation before stock changes.
- Manual fallback on low confidence.
- Unit tests for command parsing.

Do not implement backend, cloud sync, printer, WhatsApp, or GST.

First show the implementation plan and wait for my confirmation.
```

Recommended approach

Your idea is correct: **place the files in the same AI Studio chat and give the master prompt there.** But use this improved version:

1. Upload only the documents needed for the current phase.
2. Give the master prompt first.
3. Make the AI show a plan and wait.
4. Approve only one phase.
5. Build in small steps.
6. Test after every phase.
7. Start a new chat for each major phase, re-uploading the required documents.

This is better than dumping every document and asking for the full app at once, because it reduces scope creep, keeps the AI focused, and protects the most important rule: **no stock change without an auditable ledger entry.**[8]

⁂

1. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
2. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
3. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
4. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
5. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
6. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
7. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf
8. Hey-_So-I-want-to-build-a-system-that-helps-kirana.pdf